EXIGENCES DE LA PoC

Preuve de concept - orientation hospitalière
en situation d'urgence

Ce document définit les **exigences fonctionnelles, techniques et non fonctionnelles** de la preuve de concept MedHead. Il établit leur périmètre de validation, les éléments démontrés par le logiciel et les conditions nécessaires à une intégration dans la plateforme cible.

1 290	800 req/s	7
établissements simulés	objectif de charge	tests backend validés

Capacité étudiée : rechercher des établissements compatibles avec une spécialité médicale, disposant de lits simulés, situés dans un rayon donné, puis proposer des trajets lorsque le fournisseur de routage répond.

Composants : API REST Spring Boot, interface Vue.js, base H2, documentation OpenAPI, outillage de test, Docker et GitHub Actions.

Dépôt du code : https://github.com/naegel23/P11-MedHead-PoC

Dépôt d'architecture : https://github.com/naegel23/P11-MedHead-Architecture

Cadre : prototype technique autonome utilisant un jeu de données de test ; aucun système hospitalier réel ni dossier de patient n'est connecté.

Sommaire et principes de validation

01 Contexte, objectifs et périmètre

02 Architecture cible MedHead

03 Architecture réalisée pour la PoC

04 Exigences fonctionnelles

05 Interfaces et contrat d'API

06 Données, confidentialité et sécurité

07 Exigences non fonctionnelles et performance

08 Stratégie de test et critères de réception

09 Intégration, livraison et modularité

10 Bilan des exigences et perspectives

11 Justification des technologies (Java, Vue.js) — p. 13-14

12 Architecture microservices — p. 15-16

Lecture des statuts

Statut	Interprétation
Démontré	Le fonctionnement est constaté dans le périmètre et les conditions d'essai indiqués.
Partiellement démontré	La fonction existe, mais une dépendance ou un scénario empêche une validation complète.
À réaliser	L'exigence est identifiée pour la solution cible, sans mise en œuvre démontrée dans la PoC.
Non mesuré	Le résultat métier nécessite des données opérationnelles ou une campagne de mesure distincte.

L'objectif est de fournir une base de décision **traçable** : une capacité expérimentée n'est pas présentée comme une aptitude de production, et chaque limite observée devient une exigence d'industrialisation.

01 Contexte, objectifs et périmètre

MedHead réunit plusieurs acteurs du secteur médical au sein d'une plateforme commune. L'un des domaines prioritaires est le **système d'intervention d'urgence en temps réel**, chargé d'orienter une demande vers un établissement capable de la prendre en charge.

Besoin métier

À partir d'une **position géographique**, d'une **spécialité médicale** et d'un **rayon de recherche**, le service doit identifier les hôpitaux éligibles selon leur disponibilité et aider à retenir une destination pertinente. Le modèle cible prévoit également un échange événementiel permettant de déclencher des actions associées à l'orientation.

Objectifs mesurables

Référence	Exigence / indicateur	Validation par la PoC
OBJ-01	Acheminer plus de 90 % des urgences vers l'établissement compétent le plus proche.	Non mesuré : requiert des cas réels et une vérité métier contrôlée.
OBJ-02	Réduire le délai moyen de prise en charge de 18,25 à 12,00 minutes.	Non mesuré : exige des parcours opérationnels réels.
OBJ-03	Répondre en moins de 200 ms jusqu'à 800 requêtes/s par instance de service.	Démontré sur un endpoint backend local, hors routage externe.
OBJ-04	Fournir un contrat API, des tests, une CI et des composants réutilisables.	Démontré en partie : code, contrats et CI disponibles ; production non déployée.

Périmètre retenu

- Recherche et filtrage d'établissements, sur **données hospitalières simulées**.
- API documentée et consommée par une interface web.
- Tests backend et frontend, exécution Docker locale, campagne de performance.
- Identification des écarts de sécurité, d'événementiel et d'exploitation avant industrialisation.

Hors périmètre : accès aux dossiers de patients, intégration aux systèmes cliniques, réservation réelle d'un lit et décision médicale automatisée.

02 Architecture cible MedHead

L'architecture cible repose sur une plateforme distribuée découpée par domaines métier, comportant un niveau d'accès, des services spécialisés et un système d'intégration par événements. La capacité d'intervention d'urgence doit s'y inscrire tout en maîtrisant les contraintes de disponibilité, de sécurité et de latence.

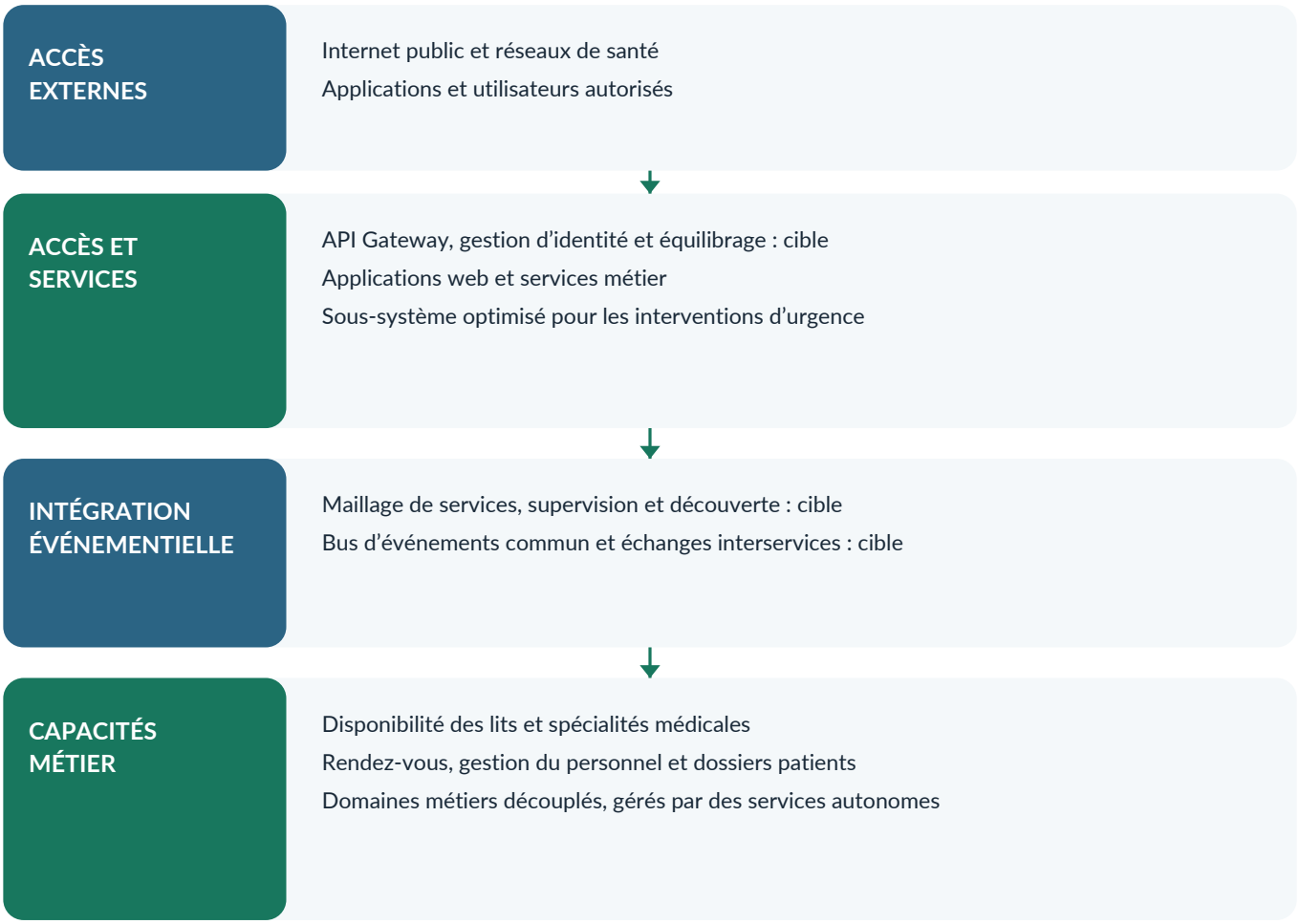


Figure 1 — Vue fonctionnelle simplifiée des couches de l'architecture cible MedHead.

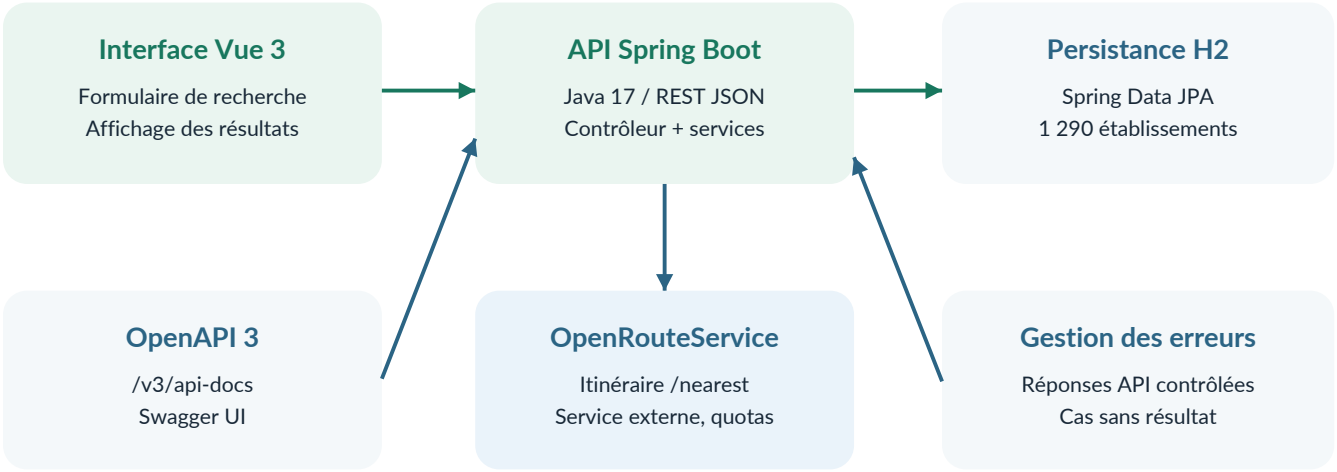
Principes structurants

- **Découplage :** interfaces contractuelles et responsabilités métier délimitées.
- **Résilience :** maîtrise des pannes, du routage et des modes dégradés.
- **Intégration :** publication d'événements et interaction avec les capacités métier de la plateforme.
- **Sécurité et observabilité :** gestion des accès, supervision, traces et alertes.

La preuve de concept se concentre sur **un bloc fonctionnel de l'intervention d'urgence**. Elle ne prétend pas fournir l'ensemble de ces composants d'infrastructure.

03 Architecture de la PoC

La réalisation comprend un **frontend Vue.js** et un **service backend Spring Boot** organisé en couches : contrôleur HTTP, services métier, repository et modèle de données. La persistance utilise une base H2 alimentée par un jeu SQL. Le calcul d'itinéraire s'appuie sur un prestataire de routage externe.



Industrialisation du code

GitHub Actions : compilation, tests backend et frontend, rapports et artefacts

Docker : construction et exécution locale du service backend

Architecture modulaire multicouche ; un service backend principal, sans bus d'événements en service.

Figure 2 — Architecture logique du PoC, principales interactions et outillage transversal.

Responsabilités des composants

Élément	Fonction
Vue.js	Collecter les critères de recherche et présenter les établissements proposés.
HospitalController	Exposer les routes REST et produire les réponses HTTP.
HospitalService	Orchestrer les règles de recherche et la logique d'orientation.
HospitalRepository / H2	Accéder aux hôpitaux et appliquer les filtres sur les données simulées.
OpenRouteService	Fournir les trajets via un appel externe, sous contrainte de quotas.

Limite d'architecture : la séparation en packages ne constitue pas une plateforme multi-microservices. L'API Gateway, le bus d'événements et le maillage de services appartiennent au périmètre cible.

Décomposition cible et conditions d'autonomie : section 12, pages 15-16.

04 Exigences fonctionnelles

Les exigences ci-dessous décrivent le comportement attendu de l'application et les éléments établis par les essais disponibles. Les valeurs de lits et les affectations de spécialités sont des **attributs simulés**, pas des données opérationnelles en temps réel.

ID	Exigence	État et justification
F-01	Accepter une localisation, une spécialité et un rayon.	**Démontré.** Paramètres disponibles dans l'interface et l'API.
F-02	Lister les établissements hospitaliers du jeu d'essai.	**Démontré.** Jeu H2 de 1 290 établissements.
F-03	Filtrer par spécialité et disponibilité des lits.	**Démontré.** Recherche et endpoints correspondants.
F-04	Restreindre les établissements au rayon demandé.	**Démontré.** Recherche géographique et endpoint de benchmark.
F-05	Évaluer un trajet et proposer une destination.	**Partiellement démontré.** Calcul tributaire du routage externe et de ses quotas.
F-06	Présenter les résultats dans une interface web.	**Démontré.** Parcours Vue → API exercé localement.
F-07	Gérer l'absence de résultat et les erreurs de service.	**À renforcer.** Cas d'erreur prévus ; gestion dégradée et résilience à compléter.
F-08	Publier un événement de réservation / attribution.	**À réaliser.** Pas de bus d'événements intégré à la PoC.

Scénario métier de référence

Étant donné une urgence localisée et nécessitant une spécialité particulière, **lorsque** l'utilisateur lance la recherche, **alors** le système doit retourner les établissements du jeu de test satisfaisant les critères de spécialité, de lits simulés disponibles et de proximité. La détermination d'un trajet dépend de la disponibilité du fournisseur externe.

Critères d'acceptation fonctionnelle

- Résultats cohérents avec les paramètres fournis et le jeu de test connu.
- Comportement explicite lorsque aucun établissement n'est éligible.
- Code HTTP et structure JSON stables, consultables dans le contrat OpenAPI.
- Message identifiable en cas d'échec du fournisseur externe.

05 Interfaces et contrat d'API

L'interface de la PoC suit un modèle **client–serveur REST**. Les appels sont indépendants sur le plan applicatif et reposent sur des ressources HTTP et des échanges JSON. Les routes exposées doivent rester documentées et testables sans dépendre de l'interface web.

Endpoints métier

Méthode / route	Objet du contrat
GET /hospital	Lister les établissements.
GET /hospital/{id}	Consulter un établissement identifié.
GET /hospital/speciality/{spec}	Filtrer par spécialité médicale.
GET /hospital/free/speciality/{spec}	Filtrer par spécialité et présence de lits libres.
GET /hospital/free/speciality/{spec}/range/{lon}/{lat}/{radius}	Filtrer selon spécialité, disponibilité et rayon.
GET /nearest/{lon}/{lat}/{radius}/{spec}	Déclencher la recommandation fondée sur le routage externe.

Documentation et interopérabilité

Interface	Adresse locale
Spécification OpenAPI 3	http://localhost:9000/v3/api-docs
Documentation Swagger UI	http://localhost:9000/swagger-ui/index.html
Point d'entrée de l'API	http://localhost:9000

Les deux interfaces de documentation ont retourné une réponse **HTTP 200** lors de la validation du backend. Ces URL correspondent à l'environnement local ; elles ne désignent pas un service hébergé publiquement.

Règles du contrat

- Paramètres **longitude, latitude, rayon et spécialité** documentés et contrôlés.
- Réponses JSON compréhensibles, y compris en l'absence d'établissement compatible.
- Codes HTTP adaptés au résultat de l'opération et aux erreurs externes.
- Contrat OpenAPI maintenu en cohérence avec le comportement réellement exposé.

06 Données, confidentialité et sécurité

Jeu de données et accès

Le service utilise une base H2 contenant **1 290 établissements**. Les noms, adresses et coordonnées proviennent d'un jeu hospitalier attribué au NHS ; les spécialités sont simulées à partir d'une nomenclature de référence et les lits disponibles sont représentés par une valeur fictive comprise entre **0 et 150**. Aucune donnée personnelle de patient n'est nécessaire au scénario logiciel étudié.

Source CSV renseignée pour la constitution de ce jeu : https://assets.nhs.uk/data/foi/Hospital.csv. La disponibilité actuelle et l'intégrité de cette source ne sont pas établies par les essais du PoC.

Exigences de sécurité

ID	Mesure attendue	État
S-01	Ne pas utiliser de données réelles de patients dans les essais.	**Démontré sur le périmètre décrit.** Dataset hospitalier simulé.
S-02	Authentifier les clients et limiter les droits d'accès aux API.	**À réaliser.** Contrôles complets non intégrés.
S-03	Protéger les échanges avec HTTPS et les mécanismes réseau adaptés.	**À réaliser.** Environnement local HTTP.
S-04	Protéger les secrets et clés des services externes.	**À renforcer.** Gestion centralisée des secrets à prévoir.
S-05	Tracer les accès, détecter les anomalies et suivre les dépendances.	**À réaliser.** Supervision de production non installée.

Règles d'industrialisation

- Appliquer une authentification et une autorisation adaptées aux utilisateurs médicaux et aux services.
- Stocker les secrets hors du code et contrôler les droits d'accès aux environnements.
- Formaliser l'analyse de conformité réglementaire si des données personnelles sont introduites.
- Conserver des journaux d'audit, avec une politique de conservation et de protection appropriée.

Principe : l'absence de dossier patient dans le PoC réduit son exposition mais ne dispense pas la solution cible des exigences de sécurité, de confidentialité et de traçabilité.

07 Exigences non fonctionnelles

Performance et comportement sous charge

La contrainte de référence vise un temps de réponse inférieur à **200 ms** jusqu'à **800 requêtes par seconde par instance**. Une campagne Apache JMeter a sollicité un endpoint de recherche locale **sans appeler le fournisseur externe de routage**.

Conditions	Résultats mesurés
100 utilisateurs virtuels ; 1 000 boucles chacun	100 000 requêtes HTTP.
Débit visé : 800 requêtes/s	Débit observé : 998,93 requêtes/s.
Erreur HTTP	0 % sur le scénario mesuré.
Temps moyen	3,98 ms.
Médiane / P90 / P95 / P99	2 ms / 5 ms / 15,95 ms / 50 ms.
Maximum observé	137 ms.

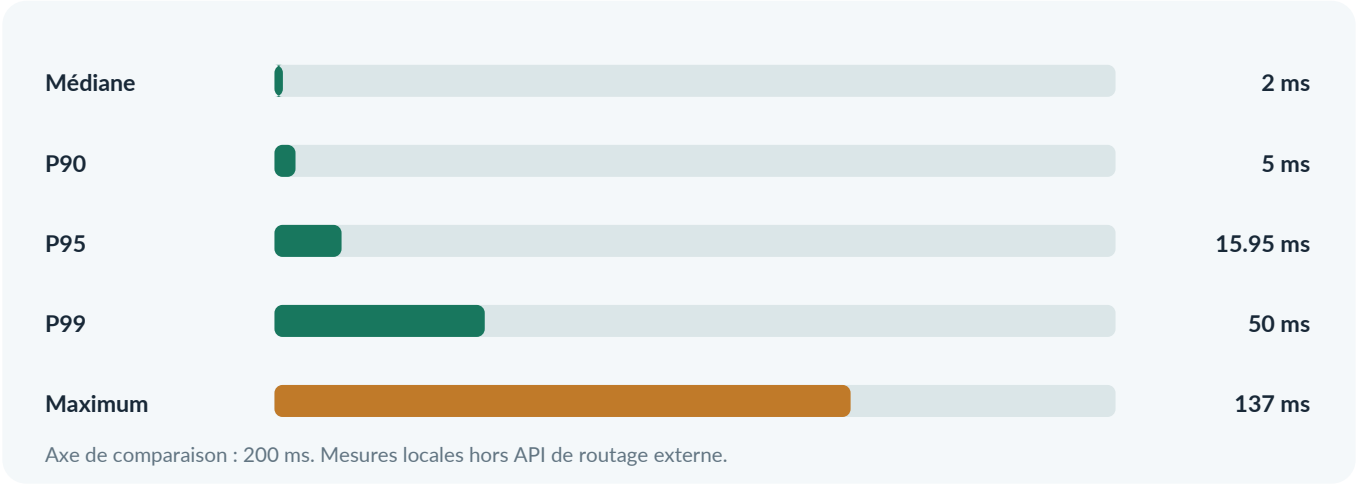


Figure 3 — Distribution des temps de réponse du test local de recherche hospitalière.

Résilience, disponibilité et dépendances

L'appel de recommandation **/nearest** dépend d'OpenRouteService. Des limites de quota ont conduit à observer des réponses fournisseur **HTTP 429** ; le backend peut alors produire une erreur. Cette situation empêche d'étendre le résultat de charge local à l'ensemble du parcours métier.

- Définir quotas et engagements de service compatibles avec le trafic attendu.
- Mettre en œuvre timeout, retry borné avec backoff, cache approprié et mode dégradé.
- Réaliser des campagnes de charge sur infrastructure représentative et sur le parcours complet.

08 Stratégie de test et critères de réception

La stratégie associe contrôles unitaires, intégration API, vérifications fonctionnelles et campagne de performance. Les essais sont reproductibles à partir de l'application, du dataset H2 et des scripts de test disponibles.

Type	Vérification	Constat
Unitaires et intégration	Filtres de spécialité, disponibilité, logique de service et endpoints.	**7 tests backend** validés par Maven.
Frontend automatisé	Formulaire et recherche avec réponses backend simulées.	Tests Vitest et Vue Test Utils exécutés.
Parcours fonctionnel	Vue → API → rendu d'établissements en local.	HTTP 200 et **20 résultats** sur le scénario contrôlé.
Conteneurisation	Construire et lancer le backend avec Docker.	API accessible, réponse HTTP 200.
Charge	Rechercher des hôpitaux avec spécialité, disponibilité et rayon.	100 000 requêtes, 0 % d'erreur sur le run de référence.
Contrat	Accéder à OpenAPI et à Swagger UI.	HTTP 200 obtenu sur les deux routes.

Vérifications à étendre

- Automatiser une suite E2E navigateur avec le backend exécuté et le jeu de données contrôlé.
- Couvrir explicitement les entrées invalides, données absentes et erreurs de services externes.
- Ajouter des tests de sécurité et de résilience, ainsi que des rapports de couverture exploités.
- Mesurer les KPI métier à partir de scénarios et d'indicateurs opérationnels représentatifs.

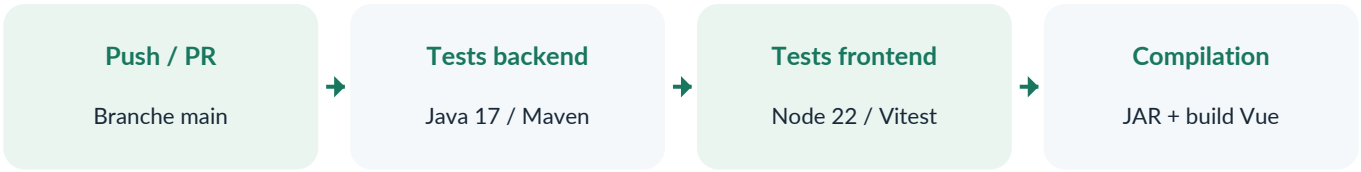
Preuves attendues pour une réception technique

Chaque exécution retenue doit permettre de retrouver le **commit Git**, les paramètres du scénario, les rapports de tests, les codes de retour, les percentiles de performance et les écarts identifiés. Une réussite ponctuelle en environnement local n'emporte pas qualification de production.

09 Intégration, livraison et modularité

Chaîne d'intégration continue

Le projet utilise **GitHub Actions** pour valider les changements sur la branche **main**, notamment lors des push et pull requests. La CI prépare l'environnement Java 17 et Node.js 22, exécute les contrôles backend et frontend et produit des artefacts de compilation.



Sorties de la CI :

rapports de tests, journaux, artefacts de compilation ; aucun déploiement en production revendiqué.

Figure 4 — Chaîne CI : contrôles automatiques et artefacts de compilation.

Exigences de livraison

ID	Exigence	Portée
I-01	Dépôt Git partagé avec code et documentation.	Démontré.
I-02	Tests automatisés au fil des intégrations.	Démontré par la CI.
I-03	Compilation reproductible et artefacts vérifiables.	Démontré dans le périmètre des builds.
I-04	Service backend conteneurisé.	Démontré par la construction et l'exécution Docker locales.
I-05	Publication et déploiement automatisés vers une plateforme de production.	À réaliser ; aucun environnement de production n'est revendiqué.

Modularité et évolutivité

La séparation **vue / contrôleur / service / repository / modèle**, les contrats REST OpenAPI et l'exécution conteneurisée constituent une base exploitable pour un composant réutilisable. L'évolution vers une architecture microservices distribuée nécessite toutefois des contrats, des politiques de sécurité, de la supervision et une gestion des événements formalisés.

Pour les frontières de services, la base dédiée et le contrat OpenAPI : section 12.

10 Bilan des exigences et décisions

Le PoC constitue une base technique vérifiable pour poursuivre l'étude du système d'orientation hospitalière. Son principal apport est de confirmer le fonctionnement du filtrage, de l'interface et de la chaîne de construction, tout en rendant visibles les dépendances et les exigences non encore couvertes.

Synthèse de conformité

Domaine	État	Éléments disponibles ou écart
Recherche hospitalière	Démontré	Spécialité, lits simulés, rayon et résultats API.
Contrat d'API	Démontré	REST JSON, OpenAPI 3, Swagger UI.
Interface utilisateur	Démontré	Parcours local Vue → API validé.
Performance locale	Démontré	998,93 req/s observées ; maximum 137 ms sur l'endpoint testé.
Routage de bout en bout	Partiel	Dépendance externe ; limitation HTTP 429 possible.
Sécurité et continuité	À réaliser	Authentification, HTTPS, gestion de secrets et modes dégradés.
Publication d'événements	À réaliser	Bus d'événements et réservation effective absents.
KPI métier	Non mesuré	Taux d'orientation et délai de prise en charge non évalués sur cas réels.

Conditions avant intégration à la solution cible

- **Sécuriser les accès et les échanges**, puis formaliser les politiques opérationnelles et de conformité.
- **Maîtriser la dépendance de routage** avec quotas, cache, timeout et mode dégradé.
- **Intégrer le service dans le système événementiel** et spécifier les contrats de réservation.
- **Confirmer les performances et la disponibilité** sur une infrastructure représentative.
- **Mesurer les bénéfices métier** avec données hospitalières et processus opérationnels fiables.

Conclusion : la poursuite du projet est justifiée pour la capacité technique expérimentée. L'adoption en environnement médical opérationnel est conditionnée à la résolution des écarts ci-dessus et à la validation spécifique des indicateurs métier.

Documents associés : Stratégie de test ; Solution Building Blocks ; Validation des principes d'architecture ; Reporting des résultats de la PoC.

Traçabilité : https://github.com/naegel23/P11-MedHead-PoC

11 Justification des choix technologiques

La sélection des technologies est motivée par les exigences du Consortium MedHead, le périmètre de la PoC et les critères de maintenabilité, de testabilité et d'industrialisation.

11.1 Langage backend : Java 17 et Spring Boot

Le **principe C3** du document *Principes de l'architecture* recommande de **favoriser les langages JVM**. Il s'agit d'une orientation architecturale, **et non d'une obligation exclusive d'utiliser Java**. Java, Kotlin et Scala sont des options compatibles avec cette orientation.

Comparaison des alternatives JVM

Langage	Atouts pour MedHead	Arbitrage / limites
Java 17	Écosystème serveur mature ; Spring Boot, Spring Data JPA, JUnit, Mockito, Maven et OpenAPI.	Retenu : cohérence des outils, architecture explicite et maintenance accessible.
Kotlin	Syntaxe concise, nullabilité renforcée ; interopérabilité JVM et prise en charge par Spring.	Alternative crédible ; gains de concision non déterminants sur une PoC de faible portée.
Scala	Programmation fonctionnelle et abstractions avancées ; écosystème JVM.	Capacités plus spécialisées que les besoins de filtrage, d'exposition REST et de tests.

Critères ayant conduit à retenir Java 17

- **Alignement :** choix conforme à la préférence JVM du Consortium (C3).
- **Découpage applicatif :** séparation contrôleur, service métier, repository et modèle de données.
- **Interopérabilité :** API REST JSON avec contrat documenté en OpenAPI 3.
- **Validation :** tests automatisés via JUnit 5 / Mockito et cycle Maven reproductible.
- **Industrialisation :** build Spring Boot, conteneur Docker et pipeline GitHub Actions.

Décision architecturale

Java 17 + Spring Boot 3.5.16 sont retenus pour l'adéquation au domaine, l'outillage de validation et l'intégration à la cible MedHead. Les contraintes de débit, de latence, de disponibilité et de sécurité doivent rester vérifiées par des essais et contrôles spécifiques ; le langage seul ne les garantit pas.

Sources et preuves : *Principes de l'architecture*, C3 (préférence JVM) ; *Exigences de la PoC*, §03, §05, §08 et §09 ; dépôt *P11-MedHead-PoC* (Java 17, Maven, CI GitHub Actions).

11.2 Framework frontend : Vue.js

Le frontend doit proposer un formulaire de recherche (position, spécialité, rayon), interroger l'API REST Spring Boot et afficher les établissements renvoyés. La PoC ne nécessite ni administration métier complexe ni plateforme frontend multi-modules.

Analyse des alternatives frontend

Solution	Points forts	Adéquation au périmètre PoC
Vue.js	Framework progressif ; réactivité et composants intégrés ; outillage Vite, Vitest et Vue Test Utils.	Retenu : faible complexité de démarrage et interface de recherche réactive.
React	Bibliothèque UI flexible ; large écosystème et approche par composants.	Alternative valable, mais davantage de décisions d'assemblage selon la structure souhaitée.
Angular	Framework très structuré : injection de dépendances, formulaires, HttpClient, routage.	Atouts pour un SI frontend étendu ; cadre plus large que les besoins immédiats de la PoC.

Critères déterminants pour Vue.js

- **Simplicité :** réaliser rapidement le formulaire et les vues de résultats sans surarchitecture.
- **Réactivité :** refléter les états de chargement, les critères et les réponses de l'API.
- **Modularité :** faire évoluer l'interface grâce aux composants indépendants.
- **Testabilité :** tester le rendu et les interactions avec Vitest et Vue Test Utils.
- **Découplage :** communiquer avec le backend JVM uniquement par des appels HTTP/JSON.

Éléments démontrés dans la PoC

L'interface **Vue 3** comporte un formulaire de recherche et affiche les résultats renvoyés par le service backend. Les **deux tests automatisés frontend** ont validé (1) l'affichage du formulaire et (2) l'affichage des établissements à partir d'une réponse backend simulée. La **compilation Vite** a également réussi. Ces preuves valident ce périmètre, sans présumer de la couverture E2E ou de la qualification en production.

Décision architecturale

Vue.js offre ici un compromis entre rapidité de développement, lisibilité, maintenance et évolutivité. React et Angular restent des options recevables pour une extension future ; aucune supériorité générale de performance de Vue.js n'est affirmée.

Sources et preuves : *Exigences de la PoC*, §03, §04 et §08 ; dépôt *P11-MedHead-PoC — frontend/src/App.spec.js* ; workflow *MedHead CI*, tests frontend et build.

12 Architecture microservices : trajectoire cible

La PoC démontre un **backend Java 17 / Spring Boot unique**, structuré en contrôleur, service métier et repository, avec une base H2 et un conteneur Docker. Cette modularité prépare une extraction, mais **ne prouve pas le déploiement de microservices indépendants**.

12.1 État actuel et séparation envisagée

RÉALISÉ DANS LA PoC

Une API Spring Boot, une persistance H2 partagée par ses modules internes, des endpoints REST/OpenAPI et un déploiement Docker local du backend.

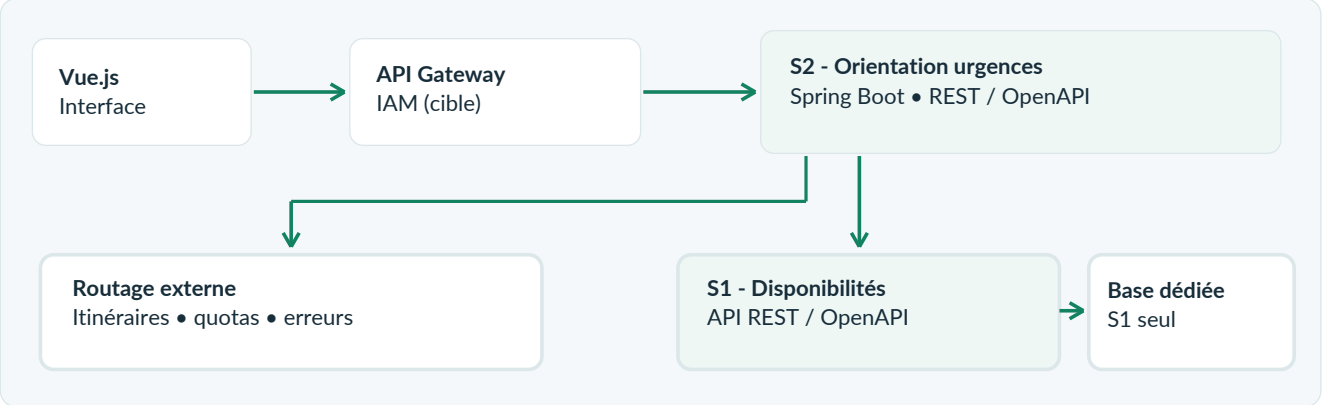
ARCHITECTURE CIBLE

Deux services métier autonomes, chacun avec un périmètre clair ; le service des disponibilités possède seul sa base. Les échanges s'effectuent par contrats explicites.

12.2 Découpage par responsabilités métier

Composant cible	Responsabilités et frontières
S1 - Disponibilités hospitalières	Référentiel des établissements, spécialités et capacités de lits. **Propriétaire unique des données hospitalières** et de leur persistance.
S2 - Orientation des urgences	Interprétation de la demande (position, spécialité, rayon), orchestration de S1 et sollicitation du fournisseur d'itinéraire. Aucun accès direct à la base de S1.
Fournisseur de routage (externe)	Calcul des trajets ; dépendance externe avec quotas, erreurs HTTP 429 et politiques de repli à définir.

12.3 Interfaces et architecture de déploiement cible



Lecture : les interfaces et bases séparées sont une **proposition d'architecture cible**, pas une fonctionnalité démontrée dans le dépôt actuel.

12.4 Microservice candidat : disponibilités hospitalières

Le premier périmètre extractible est le **service de disponibilités hospitalières (S1)** : il regroupe les données des établissements, leurs spécialités et les lits déclarés disponibles. Il constitue une frontière métier cohérente avec le rôle actuel de **HospitalRepository** et des opérations de filtrage.

Caractéristiques d'un service réellement autonome

Critère	Décision cible et preuve attendue
Périmètre métier	S1 expose les établissements, spécialités et lits disponibles. S2 consomme ses données ; aucune logique de routage externe dans S1.
Base de données dédiée	S1 possède son schéma et sa base, sans écriture SQL partagée avec S2. H2 reste une simulation PoC ; la persistance cible est à définir et valider.
Conteneur autonome	Image Docker, configuration et pipeline distincts ; déploiement, version et montée en charge de S1 indépendants de S2.
Contrat OpenAPI	Rendre explicites et versionner les endpoints de lecture, les paramètres, les réponses JSON et les erreurs ; vérification du contrat dans la CI.
Intégration et résilience	Appel REST de S2 vers S1 avec timeout, politique de repli, identifiants de corrélation et tests d'indisponibilité.
Sécurité et observabilité	Authentification/autorisation interservices, TLS, secrets externes au code ; métriques, traces et journaux à définir pour la cible.

Contrat existant réutilisable (à isoler dans S1)

Exemples d'endpoints déjà exposés : GET /hospital/{id}, **GET /hospital/free/speciality/{spec}** et **GET /hospital/free/speciality/{spec}/range/{lon}/{lat}/{radius}**. Les routes actuelles sont documentées dans **/v3/api-docs** ; leur transfert vers S1 nécessiterait une revue de contrat et des tests de non-régression.

Trajectoire de mise en œuvre et validation

1. Extraire S1 sans changer les règles de filtrage. **2.** Lui affecter une base et des migrations propres. **3.** Le construire et l'exécuter indépendamment en conteneur. **4.** Faire appeler S1 par S2 au travers de son OpenAPI, puis tester l'isolement, les erreurs et la charge.

Statut de conformité : architecture cible spécifiée ; extraction, base indépendante, déploiement séparé et interaction interservices **non démontrés à ce stade**.